

To: Alex Ametepey Teye (Teaching Assistant)

From: Ye-shua Sonpon, Team Lead, Team FiveNine (Index: 22082706)

Date: 15 August 2026

Subject: Progress Report, Campus Maintenance Service Operations Optimizer | DCIT 204/308

Project Overview

Team FiveNine is building a console-only Java application for a campus maintenance service, as part of DCIT 204/308 (Data Structures and Algorithms I & II) at the University of Ghana, Legon. The system accepts maintenance requests (e.g., broken projector, flooded bathroom), stores them in a SQLite database, prioritises them by urgency, dispatches the nearest available worker, and routes that worker using Dijkstra's algorithm.

Per course requirements, every data structure and algorithm in the assessed logic must be implemented entirely from scratch, with no use of `java.util.HashMap`, `PriorityQueue`, `ArrayList`, `Stack`, or similar standard-library collections in the core logic.

Dataset (real University of Ghana Legon campus data): 50 locations, 172 roads, 300 service requests, 30 workers/resources, all at or above required minimums.

Meetings Held

1. 2026-07-31, Zoom (Internal): Initial team meeting to divide the team into subgroups G1 to G6 and assign scope. Attendees: Ye-shua Sonpon (host), Haruna Hakeem, Erica Ama Abakah, Kobby Dua, Reginald Yeboah, Maame Esi, Gideon Foli, bakhita.

2. 2026-08-15, Google Meet (bnj-ughr-pvg): TA review meeting. TA Alex Ametepey Teye attended approximately 21:11 to 21:22. Other attendees: Ye-shua Sonpon (host), Adom Asare, Esi Twi Tawiah, Gideon Foli, Iris Cute7, Kobby Dua, Haruna Hakeem, and briefly Joy.

Implementation Status

All 15 required data structures and all 13 required algorithm categories are implemented from scratch, compile cleanly, and have been verified to work end-to-end. The full source tree (32 main classes) and test tree (20 test classes) both compile without errors.

The complete console application was run today against the live 300-request dataset, covering database loading and CSV seeding, all five routing algorithms (BFS, DFS, Dijkstra, Kruskal, Prim), request submission and dispatch with a computed route, greedy worker assignment, and a full DP table, with no crashes.

Data Structures (15, all from scratch)

DynamicArray, LinkedList, Stack, Queue, CircularQueue, Deque, PriorityQueue (min-heap), HashTable, BST, BalancedBST, BTree, CustomSet, CustomMap, Graph, DisjointSet.

Algorithms (13 required categories)

Linear search, binary search, selection sort, insertion sort, merge sort, quicksort (plus a deliberate worst-case variant for the required $O(n^2)$ trace), BFS, DFS, Dijkstra's shortest path, Kruskal's MST, Prim's MST, greedy nearest-worker assignment (with written counterexample), and 0/1 knapsack DP.

G1: Models and Database

Location.java, Road.java, ServiceRequest.java, Resource.java, and DatabaseManager.java (SQLite/JDBC, 6-table schema, CSV auto-seeding) are complete. All of this work was carried out by **Ye-shua Sonpon**. No other G1 member currently has an independent commit on these files.

G2: Linear Structures

- DynamicArray.java: Reginald Osborn Ekow Yeboah (index: 22299153)
- LinkedList.java, CircularQueue.java: Esi Twi Tawiah
- Queue.java, Deque.java: Abu-Naeem Abdul-Manan

Stack.java: An initial version by Reginald Osborn Ekow Yeboah was deleted from the branch and subsequently fully rewritten by **Ye-shua Sonpon**. The current Stack.java is entirely that rewrite.

G3: Priority Queue, Trees, and Hash Table

Adom Asare delivered the complete set: BST.java, BalancedBST.java, BTree.java, CustomMap.java, CustomSet.java, HashTable.java, PriorityQueue.java, plus matching JUnit tests for all seven.

Note: A second independent PriorityQueue.java from another contributor (seyramglory) was reviewed but not merged. It stored its heap in java.util.ArrayList, which the project bans. Adom Asare's version correctly uses our own DynamicArray.

G4: Search and Sort

SearchEngine.java and SortEngine.java were built across several contributors: **Kojo.S** (initial implementation), **Erica Ama Abakah** (SearchEngine/SortEngine build-out), and **Haruna Hakeem** (null-request handling, null sort-key validation, regression tests for edge cases).

SortEngine also includes a deliberately-bad quicksortLastPivot() method kept solely to provide the required $O(n^2)$ worst-case trace; the main quicksort uses a random pivot to avoid that case.

G5: Graph and Routing

No named G5 member has delivered this component yet. Graph.java (adjacency list and matrix, BFS, DFS, Dijkstra, Prim, Kruskal) and DisjointSet.java (union-find with path compression) were completed by **Ye-shua Sonpon** to unblock the rest of the system. Whoever is responsible for G5 still needs to read through and be able to independently defend this code.

G6: Optimisation

No named G6 member has delivered this component yet. GreedyAssignment.java (nearest-worker assignment plus a written counterexample) and DPKnapsack.java (0/1 knapsack DP for shift scheduling) were completed by **Ye-shua Sonpon**, same situation as G5.

Console Integration Layer

Main.java and seven section-menu classes wiring the whole system together were built by **Ye-shua Sonpon**. This layer was not part of the G1 to G6 subgroup agreement and represents shared integration work.

Testing Status

20 JUnit test classes exist, one per class.

11 fully written (normal, boundary, and invalid-input cases per our evidence checklist):

BalancedBSTTest, BSTTest, BTreeTest, CustomMapTest, CustomSetTest, DequeTest, HashTableTest, PriorityQueueTest, QueueTest, SearchEngineTest, SortEngineTest.

9 still placeholder stubs (deliberately failing until real assertions are written):

DatabaseManagerTest, DPKnapsackTest, GreedyAssignmentTest, DisjointSetTest, GraphTest, CircularQueueTest, LinkedListTest, StackTest, DynamicArrayTest.

Outstanding Items

The following remain to be completed before submission:

- 9 stub test classes with real assertions (DatabaseManagerTest, DPKnapsackTest, GreedyAssignmentTest, DisjointSetTest, GraphTest, CircularQueueTest, LinkedListTest, StackTest, DynamicArrayTest)
 - 6 required trace tables: binary search, insertion sort, merge sort/quicksort, Dijkstra, Kruskal or Prim, DP knapsack
 - 3 required proof sketches / loop invariants
 - 2 required counterexamples (formal writeup): the greedy failure case exists as a string in GreedyAssignment.demonstrateCounterexample() and needs to be turned into report prose; the unsorted binary search failure case has not been written up yet
 - Performance experiments: timing code works, but the full sweep (100 / 500 / 1K / 5K / 10K input sizes) has not been run and graphs have not been plotted
 - 3 algorithm parameters derived from team members' index numbers
 - Full technical report (12-section structure already outlined in the repository)
-